

Module 3: Data Exploration

The following tutorial contains examples of Python code for data exploration. You should refer to the "Data Exploration" chapter of the "Introduction to Data Mining" book (available at <https://www-users.cs.umn.edu/~kumar01/dmbook/index.php>) to understand some of the concepts introduced in this tutorial notebook. The notebook can be downloaded from <http://www.cse.nyu.edu/~atan/dmbook/tutorials/tutorial3/tutorial3.ipynb>.

Data exploration refers to the preliminary investigation of data in order to better understand its specific characteristics. There are two key motivations for data exploration:

1. To help users select the appropriate preprocessing and data analysis technique used.
2. To make use of humans' abilities to recognize patterns in the data.

Read the step-by-step instructions below carefully. To execute the code, click on the cell and press the SHIFT-ENTER keys simultaneously.

There are additional questions embedded among the code segments that you are to answer. Read carefully!

Edit your answer into the markdown blocks.

Finally run the whole script and download your file as an .html file.

3.1. Summary Statistics

Summary statistics are quantities, such as the mean and standard deviation, that capture various characteristics of a potentially large set of values with a single number or a small set of numbers. In this tutorial, we will use the Iris sample data, which contains information on 150 Iris flowers, 50 each from one of three Iris species: Setosa, Versicolour, and Virginia. Each flower is characterized by five attributes:

- sepal length in centimeters
- sepal width in centimeters
- petal length in centimeters
- petal width in centimeters
- class (Setosa, Versicolour, Virginia)

In this tutorial, you will learn how to:

- Load a CSV data file into a Pandas DataFrame object.
 - Compute various summary statistics from the DataFrame.
- To execute the sample program shown here, make sure you have installed the Pandas library (see Module 2).

1. First, you need to download the **Iris dataset** from the UCI machine learning repository.

Code: The following code uses Pandas to read the CSV file and store them in a DataFrame object named data. Next, it will display the first five rows of the data frame.

```
In [45]: import pandas as pd

data = pd.read_csv('http://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data', header=None)
data.columns = ['sepal length', 'sepal width', 'petal length', 'petal width', 'class']

data.head()
```

	sepal length	sepal width	petal length	petal width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

2. For each quantitative attribute, calculate its average, standard deviation, minimum, and maximum values.

Code:

```
In [46]: from pandas.api.types import is_numeric_dtype

for col in data.columns:
    if is_numeric_dtype(data[col]):
        print('%s: ' % (col))
        print('\t Mean = %.2f' % data[col].mean())
        print('\t Standard deviation = %.2f' % data[col].std())
        print('\t Minimum = %.2f' % data[col].min())
        print('\t Maximum = %.2f' % data[col].max())
```

sepal length:
Mean = 5.84
Standard deviation = 0.83
Minimum = 4.30
Maximum = 7.90

sepal width:
Mean = 3.05
Standard deviation = 0.43
Minimum = 2.00
Maximum = 4.40

petal length:
Mean = 3.76
Standard deviation = 1.76
Minimum = 1.00
Maximum = 6.90

petal width:
Mean = 1.20
Standard deviation = 0.76
Minimum = 0.10
Maximum = 2.50

QUESTION A: The for-loop above is iterating through the data frame. Explain why the if-statement makes this code more robust.

YOUR ANSWER: The if statement prevents any non-numeric columns from being used with mean(), std() etc.

3. For the qualitative attribute (class), count the frequency for each of its distinct values.

Code:

```
In [47]: data['class'].value_counts()
```

class	count
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

Name: count, dtype: int64

4. It is also possible to display the summary for all the attributes simultaneously in a table using the describe() function. If an attribute is quantitative, it will display its mean, standard deviation and various quantiles (including minimum, median, and maximum) values. If an attribute is qualitative, it will display its number of unique values and the top (most frequent) values.

Code:

```
In [48]: data.describe(include='all')
```

	sepal length	sepal width	petal length	petal width	class
count	150.000000	150.000000	150.000000	150.000000	150
unique	NaN	NaN	NaN	NaN	3
top	NaN	NaN	NaN	NaN	Iris-setosa
freq	NaN	NaN	NaN	NaN	50
mean	5.843333	3.054000	3.758667	1.198667	NaN
std	0.828066	0.433594	1.764420	0.763161	NaN
min	4.300000	2.000000	1.000000	0.100000	NaN
25%	5.100000	2.800000	1.600000	0.300000	NaN
50%	5.800000	3.000000	4.350000	1.300000	NaN
75%	6.400000	3.300000	5.100000	1.800000	NaN
max	7.900000	4.400000	6.900000	2.500000	NaN

Note that count refers to the number of non-missing values for each attribute.

5. For multivariate statistics, you can compute the covariance and correlation between pairs of attributes.

Code:

```
In [49]: print('Covariance:')
data.cov()
```

Covariance:

```
ValueError                                Traceback (most recent call last)
Cell In[49], line 2
      1 print('Covariance:')
----> 2 data.cov()

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/frame.py:10859, in DataFrame.cov(self, min_periods, ddof, numeric_only)
    10857 cols = data.columns
    10858 idx = cols.copy()
-> 10859 mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)
    10861 if notna(mat).all():
    10862     if min_periods is not None and min_periods > len(mat):

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/frame.py:10889, in DataFrame.to_numpy(self, dtype, copy, na_value)
    10887 if dtype is not None:
    10888     dtype = np.dtype(dtype)
-> 10889 result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
    10890 if result.dtype is not dtype:
    10891     result = np.array(result, dtype=dtype, copy=False)

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/internals/managers.py:1656, in BlockManager.as_array(self, dtype, copy, na_value)
    1654     arr.flags.writeable = False
    1655 else:
-> 1656     arr = self._interleave(dtype=dtype, na_value=na_value)
    1657     # The underlying data was copied within _interleave, so no need
    1658     # to further copy if copy=True or setting na_value
    1660 if na_value is lib.no_default:

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/internals/managers.py:1715, in BlockManager._interleave(self, dtype, na_value)
    1713     else:
    1714         arr = blk.get_values(dtype)
-> 1715         result[r1.indexer] = arr
    1716         itemmask[r1.indexer] = 1
    1718 if not itemmask.all():

ValueError: could not convert string to float: 'Iris-setosa'
```

Code:

```
In [50]: print('Correlation:')
data.corr()
```

Correlation:

```
ValueError                                Traceback (most recent call last)
Cell In[50], line 2
      1 print('Correlation:')
----> 2 data.corr()

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/frame.py:10704, in DataFrame.corr(self, method, min_periods, numeric_only)
    10702 cols = data.columns
    10703 idx = cols.copy()
-> 10704 mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)
    10706 if method == "pearson":
    10707     correl = libalgos.nancorr(mat, minp=min_periods)

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/frame.py:10889, in DataFrame.to_numpy(self, dtype, copy, na_value)
    10887 if dtype is not None:
    10888     dtype = np.dtype(dtype)
-> 10889 result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
    10890 if result.dtype is not dtype:
    10891     result = np.array(result, dtype=dtype, copy=False)

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/internals/managers.py:1656, in BlockManager.as_array(self, dtype, copy, na_value)
    1654     arr.flags.writeable = False
    1655 else:
-> 1656     arr = self._interleave(dtype=dtype, na_value=na_value)
    1657     # The underlying data was copied within _interleave, so no need
    1658     # to further copy if copy=True or setting na_value
    1660 if na_value is lib.no_default:

File ~/opt/anaconda3/lib/python3.11/site-packages/pandas/core/internals/managers.py:1715, in BlockManager._interleave(self, dtype, na_value)
    1713     else:
    1714         arr = blk.get_values(dtype)
-> 1715         result[r1.indexer] = arr
    1716         itemmask[r1.indexer] = 1
    1718 if not itemmask.all():

ValueError: could not convert string to float: 'Iris-setosa'
```

Below in the next section are some scatterplots of these attributes or variables.

QUESTION B: How do these correlation values correspond to the scatterplots (below) visualizing their joint distribution, in general?

YOUR ANSWER: Correlation values near 0 correspond to more random looking distributions, negative correlation values mean higher values of one variable tend to correspond to lower values of the other variable, positive correlation values mean higher values of one variable tend to correspond to higher values of the other variable, and correlation values near 1 correspond to distributions that look more obviously linear.

3.2. Data Visualization

Data visualization is the display of information in a graphic or tabular format. Successful visualization requires that the data (information) be converted into a visual format so that the characteristics of the data and the relationships among data items or attributes can be analyzed or reported.

In this tutorial, you will learn how to display the Iris data created in Section 3.1. To execute the sample program shown here, make sure you have installed the matplotlib library package (see Module 0 on how to install Python packages).

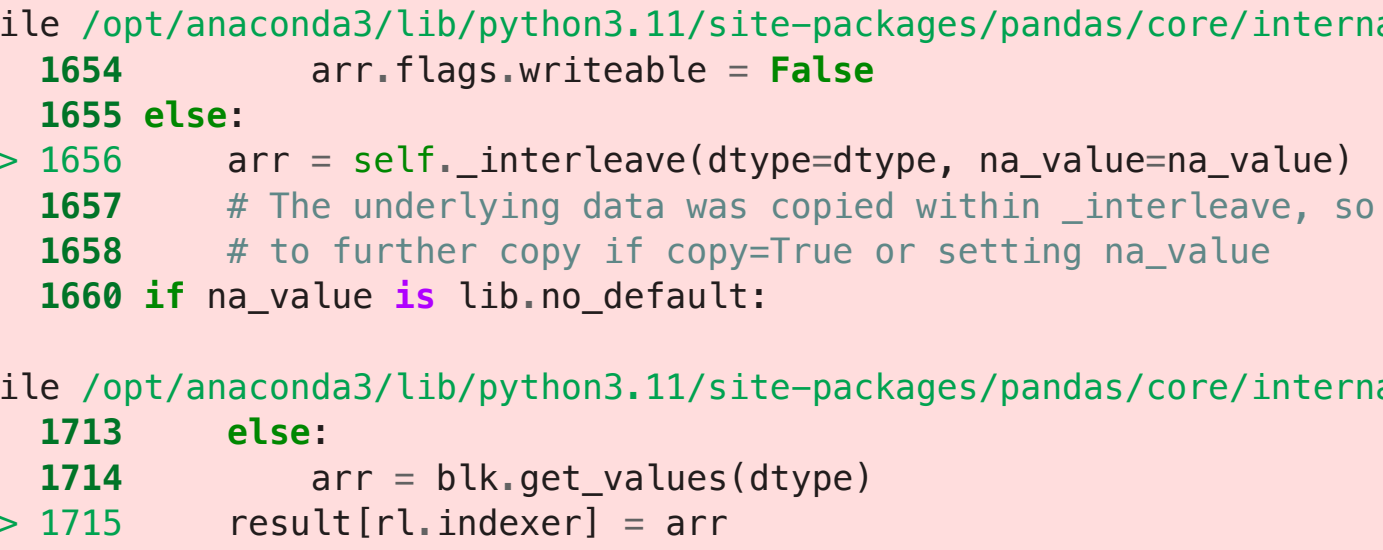
1. First, we will display the histogram for the sepal length attribute by discretizing it into 8 separate bins and counting the frequency for each bin.

Code:

```
In [51]: %matplotlib inline

data['sepal length'].hist(bins=8)
```

Out[51]: <Axes: >



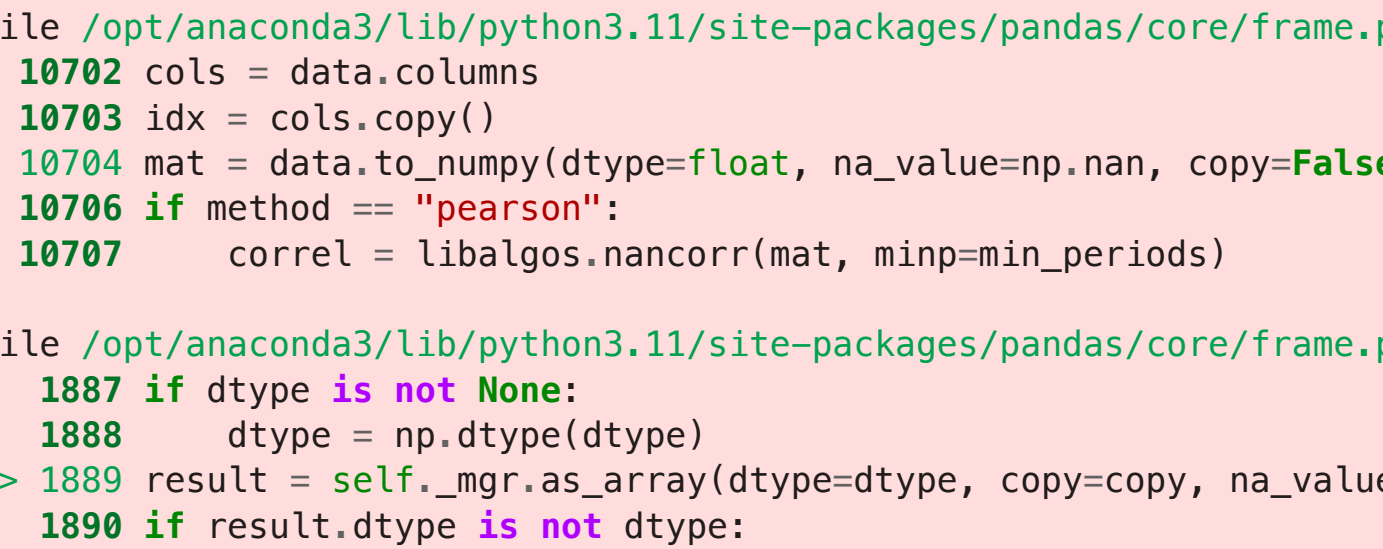
QUESTION C: How might you address the problem that a histogram depends on the number and location of the bins? Experiment with different bin numbers.

YOUR ANSWER: I don't know if this is the most sound rule but I might try adding as many bins as can be displayed without creating extra gaps in the histogram. For the sepal length histogram it seems like if you have 29 bins a gap forms between bars on the right and more gaps start forming as you go into numbers above 29. I settled for 25 since it's a multiple of 5 and it's less than 29, but maybe there's a more scientific approach to determining the appropriate number of bins.

Code:

```
In [52]: data['sepal length'].hist(bins=25)
```

Out[52]: <Axes: >

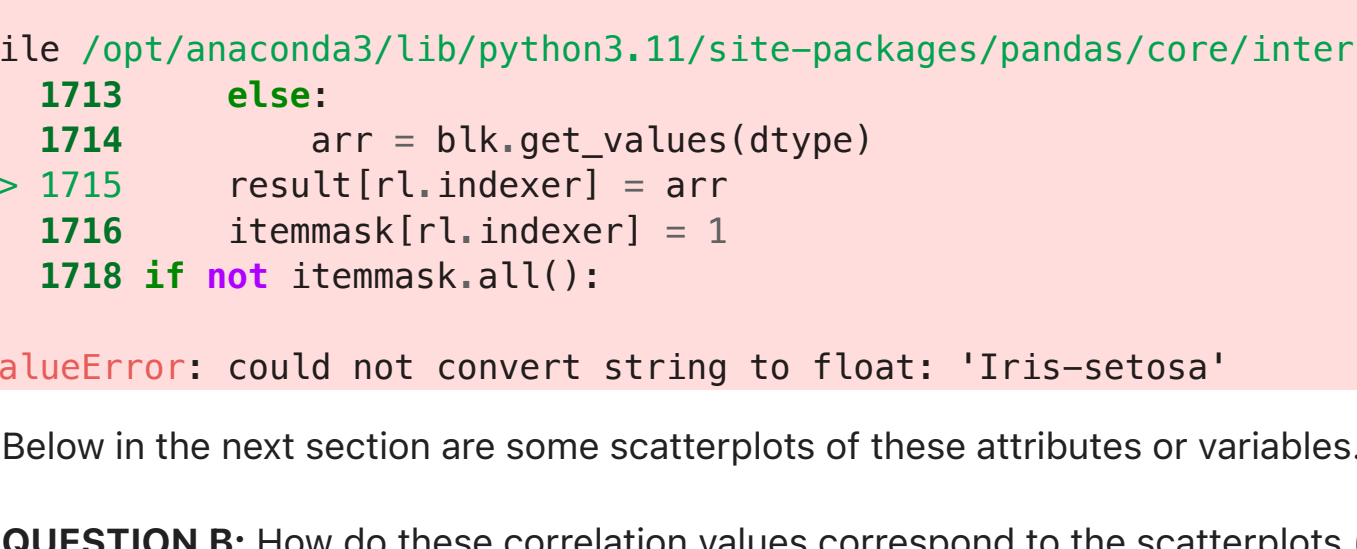


2. A boxplot can also be used to show the distribution of values for each attribute.

Code:

```
In [53]: data.boxplot()
```

Out[53]: <Axes: >



QUESTION D: Describe how a box plot can give information about whether the value of an attribute is symmetrically distributed. What can you say about the symmetry of the distributions of the attributes shown in the boxplot above?

YOUR ANSWER: The different horizontal lines that make up a box and whisker plot represent different percentiles of the distribution and the points represent outliers, so the symmetry of the plot about the median (represented by the middle horizontal line in the plot) essentially represents the symmetry of the distribution. The distribution of sepal width looks like it might be the most symmetric because it's corresponding box plot looks largely the same on either side of the median, whereas petal length might have the least symmetric distribution.

QUESTION E: Comment on the use of a box plot to explore a data set with four attributes: age, weight, height, and income.

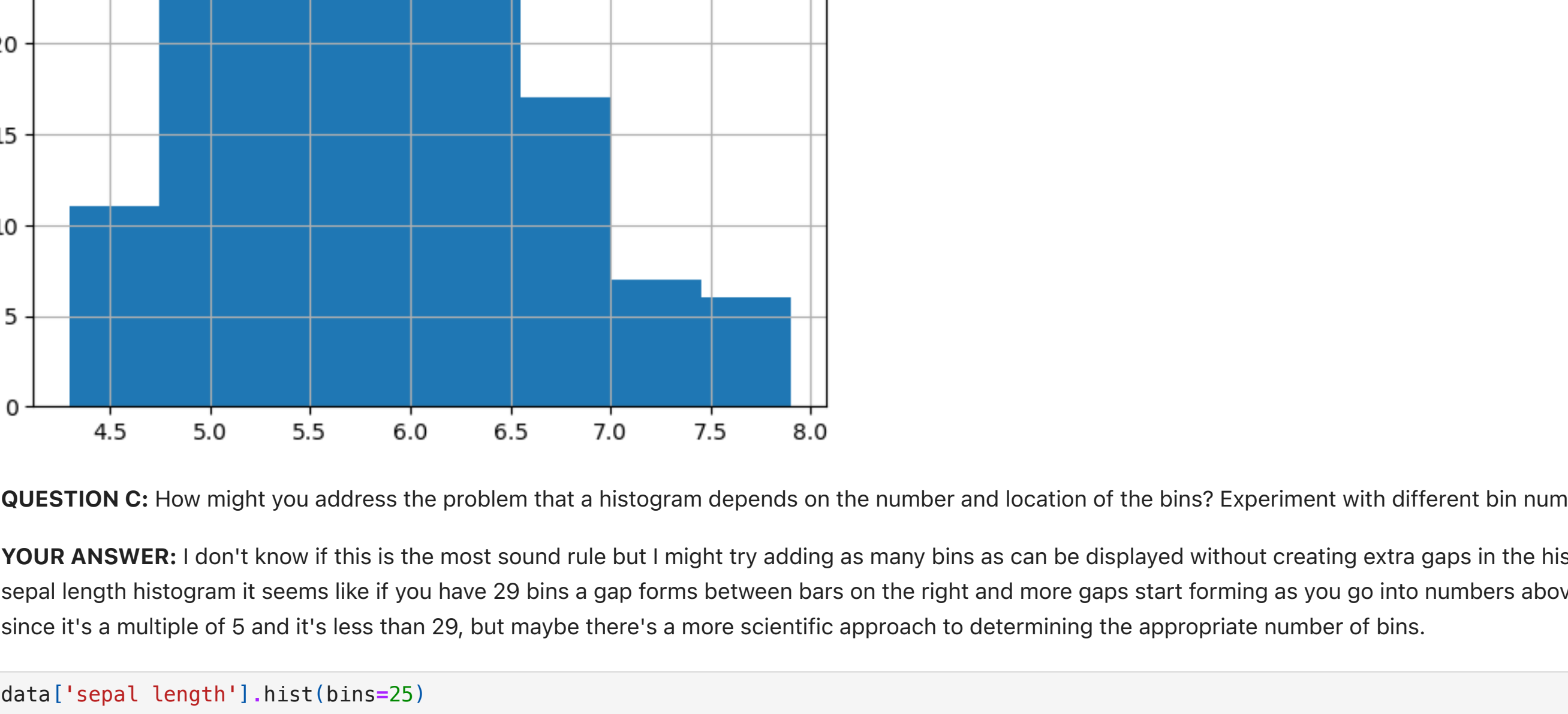
YOUR ANSWER: It would be difficult to make a graph of boxplots that contains each of those four attributes since they would all be in different units. But if you ignored units then age weight and height may all show similar distributions albeit at different ranges, since they are all physical attributes that change as a person gets older. Income also might show some relationship with age as most people don't work at all for the first several years of life and their income might change after retirement as well.

3. For each pair of attributes, we can use a scatter plot to visualize their joint distribution.

Code:

```
In [54]: import matplotlib.pyplot as plt

fig, axes = plt.subplots(3, 2, figsize=(12,12))
index = 0
for i in range(3):
    for j in range(i+1,4):
        ax1 = int(index/2)
        ax2 = index % 2
        axes[ax1][ax2].scatter(data[data.columns[i]], data[data.columns[j]], color='red')
        axes[ax1][ax2].set_xlabel(data.columns[i])
        axes[ax1][ax2].set_ylabel(data.columns[j])
        index = index + 1
```



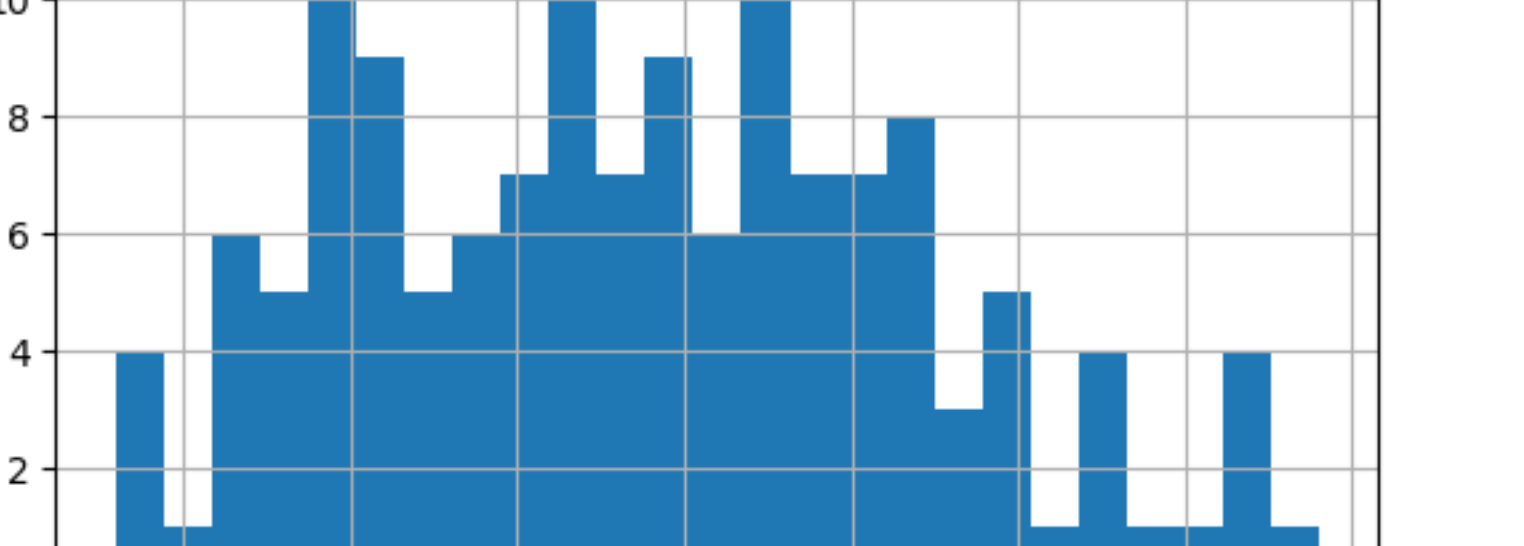
4. Parallel coordinates can be used to display all the data points simultaneously. Parallel coordinates have one coordinate axis for each attribute, but the different axes are parallel to one other instead of perpendicular, as is traditional. Furthermore, an object is represented as a line instead of as a point. In the example below, the distribution of values for each class can be identified in a separate color.

Code:

```
In [55]: from pandas.plotting import parallel_coordinates
%matplotlib inline

parallel_coordinates(data, 'class')
```

Out[55]: <Axes: >



QUESTION F: A parallel coordinates plot is an attempt to visualize a multi-dimensional data set. What problem can you identify with the such a visualization?

YOUR ANSWER: I suppose it's difficult if not impossible to identify individual cases or observations. Swapping the positions of sepal width and petal length might also make for a cleaner graph.

QUESTION G: What critique do you have with the choice of colors representing the different iris species?

YOUR ANSWER: The colors for Iris-setosa and Iris-versicolor might be a little too similar to each other.

3.3. Summary

This tutorial presents several examples for data exploration and visualization using the Pandas and matplotlib library packages available in Python.

References:

1. Documentation on Pandas. <https://pandas.pydata.org/>
2. Documentation on matplotlib. <https://matplotlib.org/>
3. Lichman, M. (2013). UCI Machine Learning Repository [http://archive.ics.uci.edu/ml]. Irvine, CA: University of California, School of Information and Computer Science.